QUERYLANG

Review 2 — IMPLEMENTATION PROGRESS REPORT

Name : Aman Katakwar

Reg no : 24BCE0160

Course Code : BCSE307P

Course : Compiler Design

Submitted to : Prof. RanjithKumar S

Overview :

QueryLang is a restricted SQL-like language and compiler-style query-processing engine. It accepts a query, converts the text into tokens, checks grammar, constructs an Abstract Syntax Tree (AST), validates table and column references, creates a relational operator plan, and evaluates that plan over an in-memory dataset. The result is either a formatted table or a diagnostic explaining why the query could not proceed.

A database query looks simple at the surface, but processing it requires several distinct operations: recognising words and operators, validating grammar, resolving table and column names, checking data types, selecting an execution plan, and producing results. A simple program that searches for keywords would not demonstrate these compiler concepts clearly. QueryLang makes each stage explicit and inspectable.

For a query such as SELECT name, cgpa FROM students WHERE cgpa >= 8.5 AND age > 20;, the lexer identifies keywords, identifiers, literals and operators. The parser validates their arrangement and builds an AST. Semantic analysis checks the schema and operand types. The planner generates relational operators, and the execution engine evaluates the plan over sample records. A valid query returns rows; an invalid query returns a stage-specific diagnostic.

The implementation uses controlled in-memory data and supports a subset of SQL-like syntax. It is an educational compiler and query runtime, not a full DBMS. Multi-table joins, aggregation, transactions, persistent storage, concurrency and cost-based optimisation should be treated as future work unless separately implemented and tested.

2. REVIEW 1 — DESIGN AND LEXER PROTOTYPE

What was completed

The supplied Review 2 report states that Review 1 established the problem definition, project scope, EBNF grammar, architecture, technology selection and a working lexical-analysis prototype. Review 1 was the design and initial implementation stage: it defined what the language should recognise and produced the first working stage of the processing pipeline.

Problem and scope definition

The initial work identified a restricted SQL-like language as a practical way to demonstrate compiler concepts using queries. The scope was kept manageable so implementation could proceed incrementally and be tested with controlled sample data instead of introducing the complexity of a production database.

Grammar and architecture planning

The EBNF grammar specified the expected structure of query statements and formed the basis for the parser added later. The architecture separated the problem into stages. Technology choices and module responsibilities were also established so the project could progress from tokenisation to full query processing.

Working lexer prototype

The principal implementation result in Review 1 was a working lexer. It scans query text and classifies parts of the input as tokens: SELECT and FROM are keywords, students and name are identifiers, 8.5 is a floating-point literal, and >= is a comparison operator. Later compiler stages can then operate on token categories rather than raw characters.

Review 1 hand-off

At the end of Review 1, the project had not yet reached a complete query-to-result pipeline. A lexer can recognise tokens, but it cannot prove that they form a valid query or that referenced columns exist. Review 2 adds the parser, AST, semantic validation, relational plan and execution engine.

3. FROM REVIEW 1 TO REVIEW 2

Main progression

The key change between the reviews is the transition from design and a lexical-analysis prototype to an integrated source-to-result system. Review 1 established the language and lexer; Review 2 implemented the stages that interpret, validate and execute a query.

Parser and AST

The parser consumes tokens according to the supported grammar and constructs an AST. The AST preserves the structure of selected columns, source table and conditions, making the query easier to analyse than a flat token list.

Schema and semantic analysis

A schema catalog records known tables, columns and data types. Semantic analysis uses this information to reject unknown names and incompatible expressions before execution.

Planner and runtime

The planner converts a valid AST into relational operators such as SCAN, FILTER, PROJECT, SORT and LIMIT. The in-memory execution engine evaluates these operators and formats the resulting rows.

Why the change matters

Each stage solves a different problem. The lexer handles characters and tokens; the parser handles grammar; semantic analysis handles names and types; the planner describes the work; and the runtime performs it. The integrated design is more complete and easier to test than isolated modules.

4. PHASE 2 ARCHITECTURE AND DATA FLOW

Stage-by-stage flow

The program passes a different representation between stages. The lexer converts characters to tokens. The parser converts tokens to an AST. Semantic analysis checks the AST against schema information. The planner creates a relational intermediate representation. The execution engine evaluates the plan and returns rows. Each stage gives the developer a clear place to inspect the query or identify an error.

Schema catalog connection

The schema catalog describes known tables, columns and data types. It enables resolution of references such as students.cgpa and rejection of columns not declared in the schema. This is the symbol-table role of the catalog.

Error path

Invalid characters belong to lexical analysis; malformed clause structure belongs to parsing; unknown tables or columns and incompatible types belong to semantic analysis; and failures during operator evaluation belong to execution. Stopping at the responsible stage makes diagnostics meaningful.

Modular design

The lexer should not evaluate predicates, the parser should not fetch records, and the execution engine should not parse raw query text. This separation supports readability, testability, debugging and module-wise explanation during viva.

5. PHASE 2 COMPONENTS — LEXICAL AND SYNTAX ANALYSIS

Lexical analysis

The lexer scans the query and groups characters into token categories. Common categories include keywords, identifiers, integer and floating-point literals, strings, Boolean literals, comparison operators, punctuation and end-of-input. Whitespace and comments are skipped while source positions are maintained for diagnostics. Keyword recognition is normally case-insensitive.

Lexical error detection

Unsupported characters should not silently become part of a token. The lexer should produce a diagnostic identifying the unexpected character and, where available, its position. Reliable token classification simplifies the parser because it can consume token types instead of repeatedly inspecting characters.

Syntax analysis

The parser uses recursive-descent procedures to recognise query clauses and expressions. It checks required keywords, column lists, conditions, statement termination and unexpected trailing tokens. The supplied Review 2 report describes it as a recursive-descent LL(1) parser, using deterministic lookahead for the supported grammar.

Precedence

Boolean conditions must be parsed consistently. In the documented grammar, AND binds more tightly than OR, while comparisons form lower-level expressions. Parentheses, where supported, permit explicit grouping. The AST must preserve this structure so execution evaluates the intended condition.

Testing these stages

Lexer tests check token types, literal values and invalid characters. Parser tests check statement structure, missing tokens and unexpected tokens. Keeping the tests separate helps identify whether a defect belongs to token recognition or grammar validation.

6. AST, SYMBOL TABLE AND SEMANTIC ANALYSIS

Abstract Syntax Tree

The AST represents grammatical structure as nodes. A query node may contain selected columns, a table name, a filter expression, ordering information and a limit. Expression nodes represent literals, column references and operators. This tree provides a consistent input to semantic analysis and planning.

Symbol table / schema catalog

A compiler symbol table associates identifiers with declaration information. QueryLang's schema catalog plays a similar role for tables, columns and types. It determines whether a table is known, whether a column belongs to it, and which type should be used for validation.

Semantic checks

Semantic analysis checks meaning after syntax is accepted. It can reject unknown tables, unknown columns and incompatible operand types. For example, SELECT salary FROM students; may be syntactically valid but should be rejected if salary is not declared in the students schema.

Why validate early

Checking before planning and execution prevents later stages from repeatedly resolving names. The planner can assume the query refers to known schema elements, and the execution engine can focus on evaluating the plan. This is a central compiler principle: detect errors as early as the available information permits.

Illustrative AST

A query such as SELECT name FROM students WHERE age > 20; can be represented as a QUERY node containing SELECT(name), FROM(students), and WHERE(GREATER_THAN(COLUMN(age), INTEGER(20))). The exact labels depend on the AST visualiser in the source code.

7. INTERMEDIATE REPRESENTATION AND EXECUTION

Relational IR

An intermediate representation separates surface syntax from execution. QueryLang uses relational operators to describe the work required to answer a query. SCAN identifies the source relation, FILTER applies a predicate, PROJECT selects columns, SORT orders rows and LIMIT restricts the result count.

Example plan

For SELECT name, cgpa FROM students WHERE cgpa >= 8.5 AND age > 20;, a representative plan is PROJECT [name, cgpa] above FILTER [cgpa >= 8.5 AND age > 20] above SCAN [students]. The plan exposes the intended sequence of operations.

In-memory execution

The execution engine evaluates the plan over sample records stored in memory. It checks rows against the predicate, retains matching records, constructs projected rows, sorts when requested and applies limits. The final result is formatted as a readable table.

Execution evidence

Where implemented, scan and filter counts can show how many records were examined and retained. Output formatting aligns headers and values. Exact row order and formatting should be verified by running the actual source rather than relying solely on illustrative output.

Scope boundary

The current design focuses on a restricted grammar and in-memory records. Joins, grouping, aggregation, persistent storage and cost-based optimisation should not be claimed as implemented without source and test evidence.

8. ERROR HANDLING AND FUNCTIONAL BEHAVIOUR

Lexical errors

These occur when input characters do not match supported token patterns. The lexer should report the issue immediately, preventing an incomplete token stream from causing confusing downstream errors.

Syntax errors

These occur when recognised tokens are arranged in an invalid form, such as a missing required keyword or malformed clause. The parser can report the expected token and the token found.

Semantic errors

These occur when a syntactically valid query references a missing table or column or uses incompatible types. They should be reported before planning and execution.

Execution errors and empty results

Execution errors occur during evaluation of an accepted plan. In contrast, a valid query returning zero rows is not an error; it means no record satisfied the condition.

Example scenarios

An unsupported character should produce a lexical diagnostic; a missing FROM clause should produce a syntax diagnostic; an unknown table or column should produce a semantic diagnostic; and a valid filter with no matches should produce an empty result.

9. PHASE 2 DELIVERABLES

Core modules

The supplied implementation brings together a lexer, parser, AST hierarchy and visualisation, schema catalog/symbol table, semantic analyzer, relational query planner, in-memory execution engine, diagnostics, command-line interface, demo routines, sample data and automated tests.

Working source code

The project is implemented in Python and uses standard-library facilities such as regular expressions, enumerations, dataclasses, abstract base classes, typing and argument parsing. The supplied code PDF remains the definitive full implementation.

Functional demonstration

The documented command-line modes are an interactive REPL, a demo mode, an automated test mode and a schema display mode. Use the actual saved filename when running these commands.

Test cases

The supplied Review 2 report documents a 16-case verification matrix and a PyUnit suite of 20 methods across lexer, parser, semantic, planner and execution tests. It covers positive queries, compound conditions, sorting, limits, lexical errors, syntax errors, unknown names, incompatible types and empty results.

Intermediate results and explanation

Intermediate results include tokens, AST structure, schema metadata, semantic outcomes, relational plans, result tables and diagnostics. The student should be able to describe the input and output of each module.

10. TESTING AND FUNCTIONAL CORRECTNESS

Testing approach

The supplied Review 2 report records 16 formal matrix cases and 20 PyUnit methods and reports zero failures and zero errors. These are source-reported results and should be reconfirmed by running the current code before the review.

Positive cases

Valid SELECT, multi-column projection, WHERE filters, AND/OR predicates, ORDER BY and LIMIT confirm the main language features.

Negative cases

Invalid characters, missing tokens, malformed statements, unknown tables, unknown columns and incompatible types confirm that the program rejects invalid input at the proper stage.

Boundary cases

A valid query that matches no records should return an empty result. End-to-end tests confirm that all stages are connected and that the result or diagnostic reaches the user.

Representative matrix

The table on the next page summarises test types and expected behaviour. It is a report summary, not a replacement for actual test output.

11. REVIEW 2 EVALUATION CRITERIA

Implementation progress

Evidence is the progression from the Review 1 lexer to parsing, AST, semantic validation, planning, execution, diagnostics and tests.

Functional correctness

Demonstrate a valid SELECT and a filtered query, then show an invalid query and explain why it is rejected.

Compiler concepts

Explain tokenisation, recursive-descent parsing, AST construction, symbol-table lookup, type checking and relational IR.

Code quality

Show how responsibilities are separated, how names and data structures are organised, and how diagnostics support debugging.

Testing quality

Run tests and explain positive, negative and boundary cases. Confirm the reported test counts in the current environment.

Problem solving and innovation

The design makes otherwise hidden query-processing stages visible and uses stage-specific errors to isolate problems.

Viva understanding

Be prepared to explain an algorithm, modify a module, debug an error and describe the internal operation of the query pipeline.

12. IMPLEMENTATION PROGRESS SUMMARY

Review 1

The supplied report says Review 1 delivered the problem definition, scope, EBNF grammar, architecture, technology selection and a working lexer prototype.

Review 2

The supplied report documents a recursive-descent LL(1) parser, typed AST, schema catalog/symbol table, semantic analyzer, relational query planner, in-memory execution engine, multi-stage diagnostics, automated verification and functional demonstrations.

What remains important

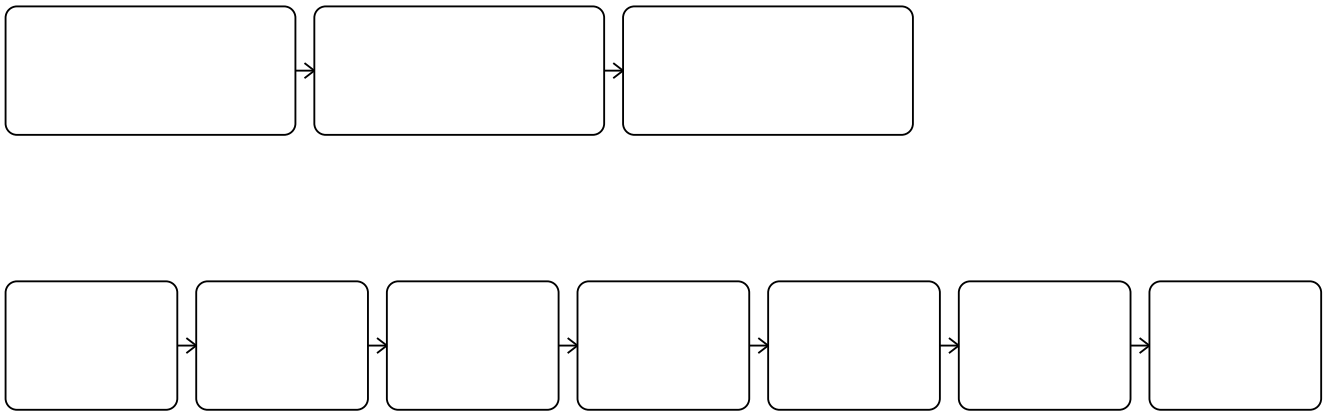
The source should be run before submission. Verify demo output, test results, schema entries and error messages. Use actual terminal captures if screenshots are required by the instructor.

Current limitations

QueryLang is an educational language processor using in-memory sample data. It should not be presented as a complete SQL implementation or production DBMS.

FIGURE A — PIPELINE COMPARISON

Review 1 focused on design and the first working lexical stage. Review 2 connects the later compiler stages into a query-to-result pipeline.



The diagram is schematic and describes the progression documented in the supplied Review 2 report.

REPRESENTATIVE TEST MATRIX

ID	Scenario	Expected behaviour
T01	Basic SELECT	Return selected columns
T02	Multi-column projection	Return requested fields only
T03	WHERE comparison	Retain matching rows
T04	AND / OR	Apply Boolean logic
T05	ORDER BY	Sort records
T06	LIMIT	Restrict result count
T07	Invalid character	Lexical diagnostic
T08	Missing required token	Syntax diagnostic
T09	Unknown table	Semantic diagnostic
T10	Unknown column	Semantic diagnostic
T11	Incompatible types	Type diagnostic
T12	No matching rows	Valid empty result
T13	End-to-end query	Result or diagnostic

The source report records 16 formal matrix tests and 20 PyUnit methods. Confirm the exact cases and their pass status by running the supplied test command.

IMPORTANT CODE 1 — TOKEN RECOGNITION

Token categories allow the parser to reason about the language through named types rather than raw characters. The patterns below are representative of the token definitions in the supplied implementation.

```
class
TokenType(Enum):
SELECT = auto()
    FROM = auto()
    WHERE = auto()
    AND = auto()
    OR = auto()
    IDENTIFIER = auto()
    INTEGER = auto()
    FLOAT = auto()
    STRING = auto()
    BOOLEAN = auto()
    EQ = auto()
    NEQ = auto()
    GT = auto()
    LT = auto()
    GTE = auto()
    LTE = auto()
    EOF = auto()

TOKEN_SPECIFICATION = [
    ('WHITESPACE', r'[ \\t\\r\\n]+'),
    ('COMMENT', r'--[^\\r\\n]*'),
    ('FLOAT', r'\\d+\\.\\d+'),
    ('INTEGER', r'\\d+'),
    ('GTE', r'>='),
    ('LTE', r'<='),
    ('NEQ', r'!='),
    ('IDENTIFIER', r'[A-Za-z_][A-Za-z0-9_]*'),
```

```
]
```

The lexer converts the source string into a sequence of typed tokens. Operator ordering matters: >= should be recognised before > so a two-character operator is not split. The complete source may include additional keywords and punctuation; use the supplied code PDF for the authoritative implementation.

IMPORTANT CODE 2 — PARSER TOKEN VALIDATION

A parser must check that the current token matches the grammar requirement. A shared consume method keeps this check and its error handling consistent across parsing functions.

```
def consume(self, expected_type,
error_msg=None):
    tok =
self.current_token
if tok.type ==
expected_type:
        self.cursor
+= 1         return
tok

    found = tok.type.name if tok.type !=
TokenType.EOF else "End of Input"     expected
= expected_type.name     message = error_msg or
(         f"Expected '{expected}', but found
'{found}' "         f"with value
{tok.value!r}."     )     raise SyntaxError(
message,         line=tok.line,
column=tok.column,         hint=f"Check syntax
around '{tok.value}'."     )
```

When the expected token is present, the parser advances and returns it. If not, it raises a syntax diagnostic with context. This prevents the parser from silently accepting malformed query structure. Confirm exact method names and signatures against the submitted source before explaining this excerpt in viva.

INTERMEDIATE RESULT — RELATIONAL PLAN

Example query

```
SELECT name, cgpa FROM students
WHERE cgpa >= 8.5 AND age > 20;
```

Representative plan

```
PROJECT [name, cgpa]
  FILTER [cgpa >= 8.5 AND age > 20]
    SCAN [students]
```

The plan represents the main work required by the query. SCAN accesses the relation, FILTER retains records satisfying the predicate, and PROJECT returns only the requested fields. This is an illustrative plan; use the output generated by the actual implementation for a formal demonstration.

Representative semantic diagnostic

```
SELECT salary FROM students;

Expected behaviour:
Semantic error — salary
is not declared in the
students schema.
```

This demonstrates the difference between syntax and semantics. The query may have a valid shape but still refer to a field that is not present in the schema catalog.

DEMONSTRATION COMMANDS

Commands documented in the supplied project materials

```
python
all_in_one_querylang.py
python
all_in_one_querylang.py
--demo python
all_in_one_querylang.py
--test python
all_in_one_querylang.py
--schema
```

REVIEW 2 CHECKLIST AND REFERENCES

Criterion	Evidence to demonstrate
Implementation progress	Show the progression from Review 1 lexer to integrated pipeline.
Functional correctness	Run valid and invalid queries and explain outputs.
Compiler concepts	Explain tokens, parser, AST, symbol table, semantic checks and IR.
Code quality	Explain module separation, naming and diagnostics.
Testing	Run tests and discuss positive, negative and boundary cases.
Problem solving	Identify which stage handles a lexical, syntax or semantic error.
Innovation	Show the inspectable AST/plan and stage-specific diagnostics.
Viva understanding	Explain an algorithm, modify a module or debug an error.

References

[1] QueryLang: A Compiler-Style Query Processing Engine — Review 2 report supplied for the project.

[2] QueryLang consolidated code listing supplied for the project.

[3] Compiler Design Laboratory Project Instruction Manual, BCSE307P, Academic Year 2026–2027.

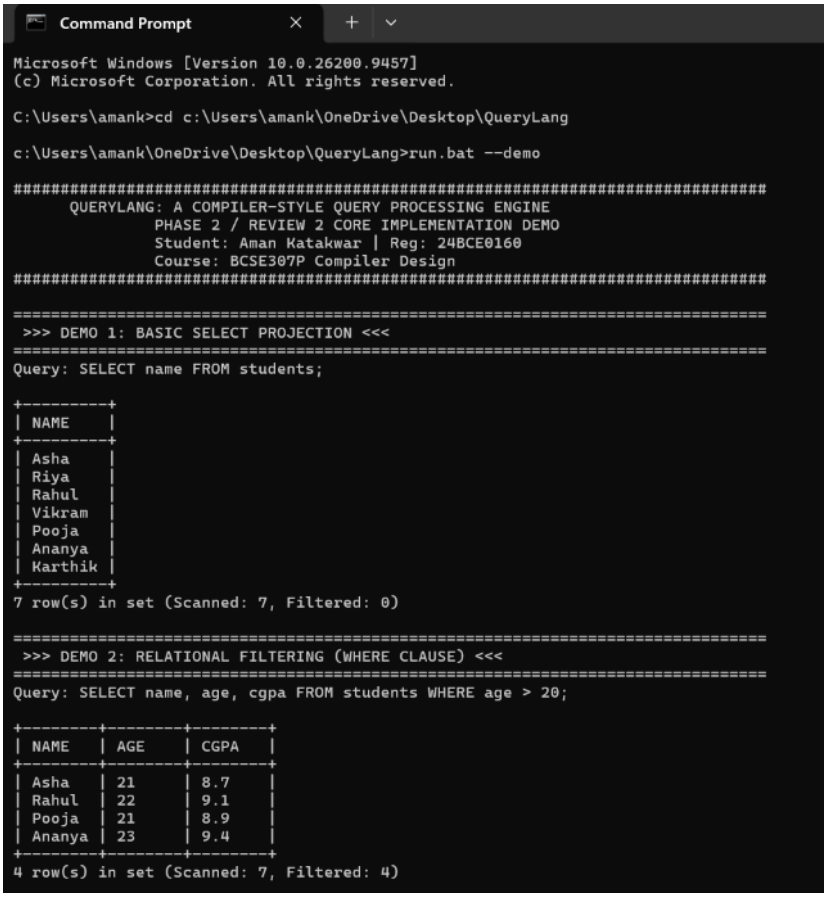
[4] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. Compilers: Principles, Techniques, and Tools, 2nd edition, Pearson.

[5] Python Software Foundation, Python 3 Documentation, https://docs.python.org/3/

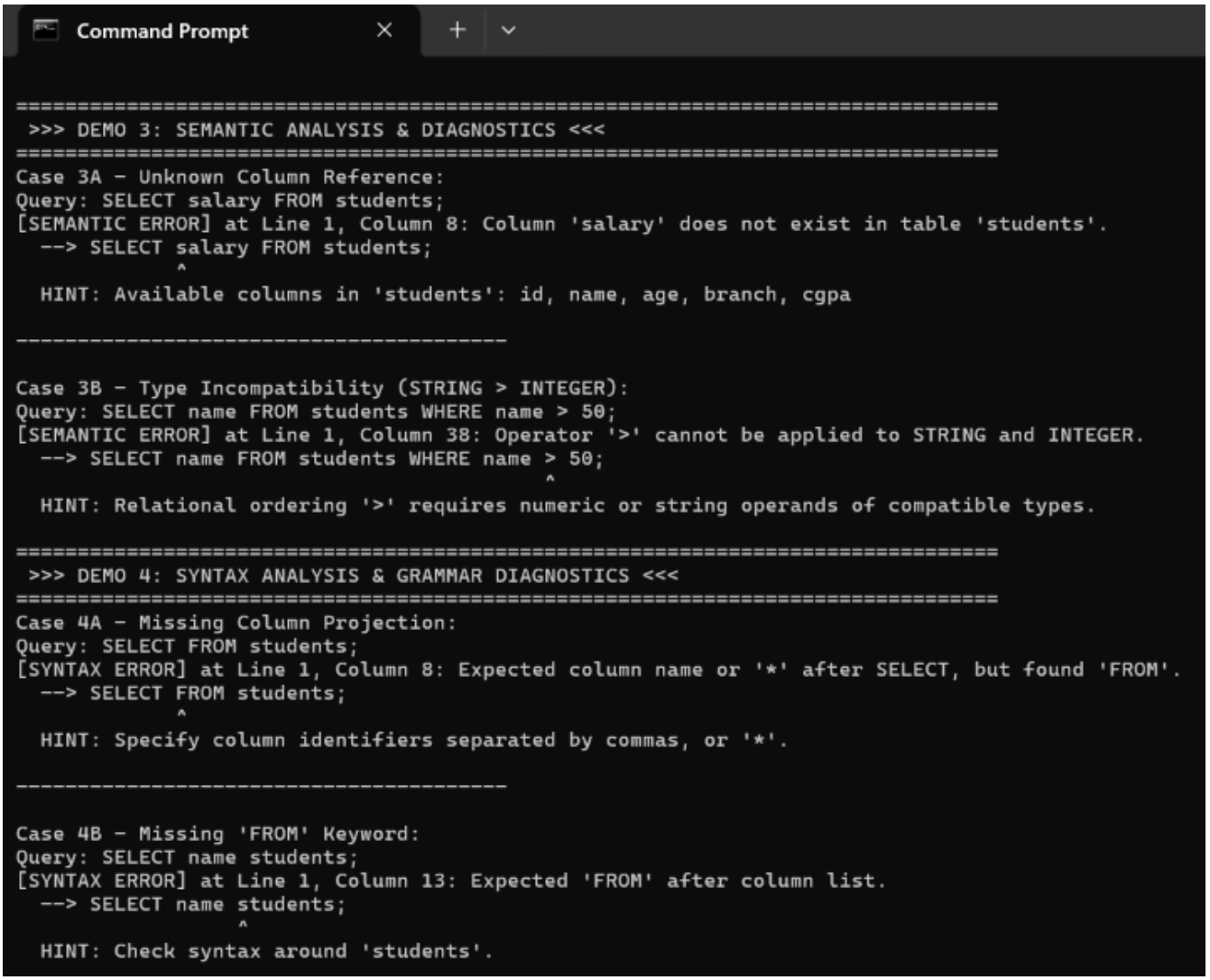
Before submission, run the source code, confirm test results, and replace illustrative output with genuine program output if screenshots are required. All fonts and diagrams in this report are black and white.

Live Demonstrations :

Demo 1 and 2 :

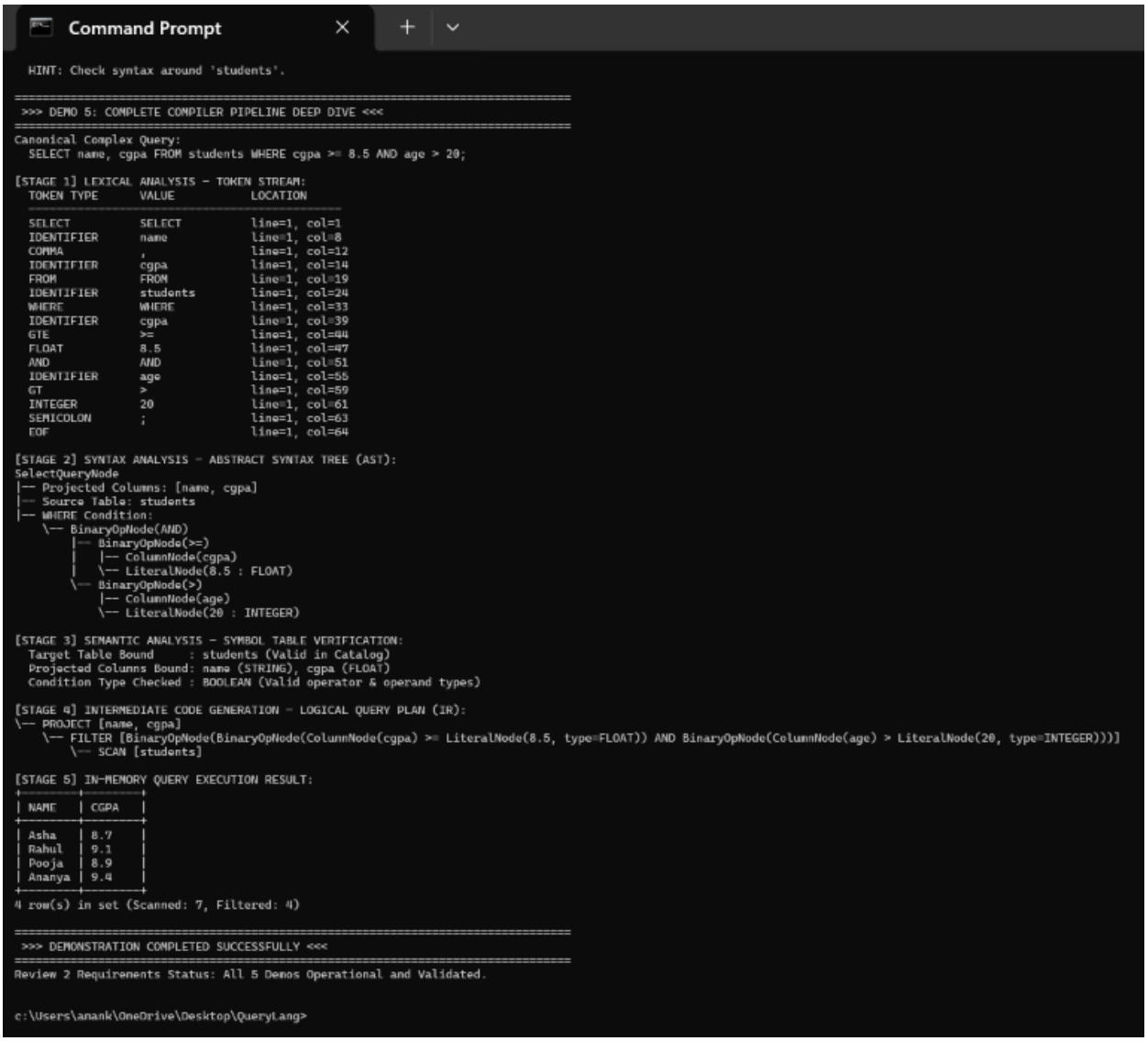


Demo 3 and 4 :


Command Prompt
==
>>> DEMO 3: SEMANTIC ANALYSIS & DIAGNOSTICS <<<
==
Case 3A - Unknown Column Reference:
Query: SELECT salary FROM students;
[SEMANTIC ERROR] at Line 1, Column 8: Column 'salary' does not exist in table 'students'.
--> SELECT salary FROM students;
^
HINT: Available columns in 'students': id, name, age, branch, cgpa
----
Case 3B - Type Incompatibility (STRING > INTEGER):
Query: SELECT name FROM students WHERE name > 50;
[SEMANTIC ERROR] at Line 1, Column 38: Operator '>' cannot be applied to STRING and INTEGER.
--> SELECT name FROM students WHERE name > 50;
^
HINT: Relational ordering '>' requires numeric or string operands of compatible types.
==
>>> DEMO 4: SYNTAX ANALYSIS & GRAMMAR DIAGNOSTICS <<<
==
Case 4A - Missing Column Projection:
Query: SELECT FROM students;
[SYNTAX ERROR] at Line 1, Column 8: Expected column name or '*' after SELECT, but found 'FROM'.
--> SELECT FROM students;
^
HINT: Specify column identifiers separated by commas, or '*'.

Case 4B - Missing 'FROM' Keyword:
Query: SELECT name students;
[SYNTAX ERROR] at Line 1, Column 13: Expected 'FROM' after column list.
--> SELECT name students;
^
HINT: Check syntax around 'students'.

Demo 5 :


Command Prompt
HINT: Check syntax around 'students'.
>>> DEMO 5: COMPLETE COMPILER PIPELINE DEEP DIVE <<<
Canonical Complex Query:
SELECT name, cgpa FROM students WHERE cgpa >= 8.5 AND age > 20;
[STAGE 1] LEXICAL ANALYSIS - TOKEN STREAM:
TOKEN TYPE VALUE LOCATION
SELECT SELECT line=1, col=1
IDENTIFIER name line=1, col=8
COMMA , line=1, col=12
IDENTIFIER cgpa line=1, col=14
FROM FROM line=1, col=19
IDENTIFIER students line=1, col=24
WHERE WHERE line=1, col=33
IDENTIFIER cgpa line=1, col=39
GTE >= line=1, col=44
FLOAT 8.5 line=1, col=47
AND AND line=1, col=51
IDENTIFIER age line=1, col=55
GT > line=1, col=59
INTEGER 20 line=1, col=61
SEMICOLON ; line=1, col=63
EOF line=1, col=64
[STAGE 2] SYNTAX ANALYSIS - ABSTRACT SYNTAX TREE (AST):
SelectQueryNode
|-- Projected Columns: [name, cgpa]
|-- Source Table: students
|-- WHERE Condition:
\-- BinaryOpNode(AND)
|-- BinaryOpNode(>=)
| |-- ColumnNode(cgpa)
| \-- LiteralNode(8.5 : FLOAT)
\-- BinaryOpNode(>)
|-- ColumnNode(age)
\-- LiteralNode(20 : INTEGER)
[STAGE 3] SEMANTIC ANALYSIS - SYMBOL TABLE VERIFICATION:
Target Table Bound : students (Valid in Catalog)
Projected Columns Bound: name (STRING), cgpa (FLOAT)
Condition Type Checked : BOOLEAN (Valid operator & operand types)
[STAGE 4] INTERMEDIATE CODE GENERATION - LOGICAL QUERY PLAN (IR):
\-- PROJECT [name, cgpa]
\-- FILTER [BinaryOpNode(BinaryOpNode(ColumnNode(cgpa) >= LiteralNode(8.5, type=FLOAT)) AND BinaryOpNode(ColumnNode(age) > LiteralNode(20, type=INTEGER)))]
\-- SCAN [students]
[STAGE 5] IN-MEMORY QUERY EXECUTION RESULT:
NAME CGPA
Asha 8.7
Rahul 9.1
Pooja 8.9
Ananya 9.4
4 row(s) in set (Scanned: 7, Filtered: 4)
>>> DEMONSTRATION COMPLETED SUCCESSFULLY <<<
Review 2 Requirements Status: All 5 Demos Operational and Validated.
c:\Users\anank\OneDrive\Desktop\QueryLang>

Formal Tests :

```
c:\Users\anank\OneDrive\Desktop\QueryLang>run.bat --test
================================================================================
      QUERYLANG COMPILER ENGINE - REVIEW 2 VERIFICATION MATRIX
================================================================================
ID     | Test Description                | Expected Behavior          | Status
--------------------------------------------------------------------------------
T-01   | Basic SELECT Projection         | Pass (7 rows returned)     | [ PASS ]
T-02   | Multi-column Projection         | Pass (2 cols, 7 rows)      | [ PASS ]
T-03   | WHERE Relational Filter         | Pass (4 rows matched)      | [ PASS ]
T-04   | Logical AND Compound Filter     | Pass (4 rows matched)      | [ PASS ]
T-05   | Logical OR Compound Filter      | Pass (2 rows matched)      | [ PASS ]
T-06   | Wildcard Projection (*)         | Pass (5 cols, 2 rows)      | [ PASS ]
T-07   | ORDER BY Ascending Sort         | Pass (sorted asc)          | [ PASS ]
T-08   | LIMIT Clause Truncation         | Pass (3 rows returned)     | [ PASS ]
T-09   | Lexical Error Detection         | LexicalError raised        | [ PASS ]
T-10   | Syntax Error (Missing Col)      | SyntaxError raised         | [ PASS ]
T-11   | Syntax Error (Missing FROM)     | SyntaxError raised         | [ PASS ]
T-12   | Semantic Error (Unknown Table)  | SemanticError raised       | [ PASS ]
T-13   | Semantic Error (Unknown Col)    | SemanticError raised       | [ PASS ]
T-14   | Semantic Error (Type Mismatch)  | SemanticError raised       | [ PASS ]
T-15   | Boundary (Empty Result Set)     | Pass (0 rows returned)     | [ PASS ]
T-16   | Pipeline Inspection Stages      | Pass (5 stages verified)   | [ PASS ]
--------------------------------------------------------------------------------
Total Verification Matrix Tests: 16 | Passed: 16 | Failed: 0
================================================================================

Running Formal PyUnit Test Suites...
test_and_condition (test_execution.TestExecution.test_and_condition) ... ok
test_basic_select (test_execution.TestExecution.test_basic_select) ... ok
test_empty_result_boundary (test_execution.TestExecution.test_empty_result_boundary) ... ok
test_filtered_select (test_execution.TestExecution.test_filtered_select) ... ok
test_or_condition (test_execution.TestExecution.test_or_condition) ... ok
test_order_by_and_limit (test_execution.TestExecution.test_order_by_and_limit) ... ok
test_basic_tokenization (test_lexer.TestLexer.test_basic_tokenization) ... ok
test_invalid_character_error (test_lexer.TestLexer.test_invalid_character_error) ... ok
test_operators_and_literals (test_lexer.TestLexer.test_operators_and_literals) ... ok
test_parse_simple_select (test_parser.TestParser.test_parse_simple_select) ... ok
test_parse_where_condition (test_parser.TestParser.test_parse_where_condition) ... ok
test_syntax_error_missing_column (test_parser.TestParser.test_syntax_error_missing_column) ... ok
test_syntax_error_missing_from (test_parser.TestParser.test_syntax_error_missing_from) ... ok
test_syntax_error_missing_semicolon (test_parser.TestParser.test_syntax_error_missing_semicolon) ... ok
test_plan_generation_scan_project (test_planner.TestPlanner.test_plan_generation_scan_project) ... ok
test_plan_generation_with_filter (test_planner.TestPlanner.test_plan_generation_with_filter) ... ok
test_semantic_error_type_mismatch (test_semantic.TestSemanticAnalyzer.test_semantic_error_type_mismatch) ... ok
test_semantic_error_unknown_column (test_semantic.TestSemanticAnalyzer.test_semantic_error_unknown_column) ... ok
test_semantic_error_unknown_table (test_semantic.TestSemanticAnalyzer.test_semantic_error_unknown_table) ... ok
test_valid_query_semantics (test_semantic.TestSemanticAnalyzer.test_valid_query_semantics) ... ok

----------------------------------------------------------------------
Ran 20 tests in 0.011s

OK

c:\Users\anank\OneDrive\Desktop\QueryLang>
```

Interactive Query Shell :

```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\amank>cd c:\Users\amank\OneDrive\Desktop\QueryLang

c:\Users\amank\OneDrive\Desktop\QueryLang>run.bat

================================================================================
   A Compiler-Style Query Processing Engine | Review 2 Core Implementation
   Student: Aman Katakwar (24BCE0160) | Guide: Prof. Ranjithkumar S
================================================================================
Commands:
  \help          - Show this help message
  \schema        - Display Schema Catalog (Symbol Table)
  \inspect <sql> - Run query with full stage-by-stage compiler inspection
  \demo          - Run the Review 2 demonstration suite
  \test          - Run the comprehensive 16-test verification matrix
  \quit or \exit - Exit the shell
================================================================================

Interactive Shell ready. Inspect mode is OFF.
Type queries terminating with a semicolon ';' or special commands.

QueryLang> \schema

=== QueryLang Schema Catalog (Symbol Table) ===
Table: students
  Column          Type         Description
  --------------- ------------ -------------------------
  id              INTEGER      Unique student identifier
  name            STRING       Full name of the student
  age             INTEGER      Age in years
  branch          STRING       Academic department
  cgpa            FLOAT        Cumulative Grade Point Average

Table: departments
  Column          Type         Description
  --------------- ------------ -------------------------
  dept_id         INTEGER      Department ID
  dept_name       STRING       Name of the department
  hod             STRING       Head of department name
  building        STRING       Campus location

QueryLang>
```